

Scalable Fantasy Baseball Draft Optimization: Exact ADP-Aware ILP and Opportunity-Cost Greedy Heuristics

Group 4

B13705051 周孟承 B13902103 鄭宇宏
B13303153 詹舒宇 B11705039 李盈盈

Abstract

We study fantasy baseball snake-draft roster construction as an operations research problem. The decision maker must select one player at each owned pick, assign drafted players to roster positions, and account for player availability implied by average draft position (ADP). We formulate an ADP-aware integer linear program (ILP) as an exact benchmark. Because this model becomes large as the player pool and roster size grow, we propose Opportunity Cost Greedy, a scalable heuristic that drafts where delaying to the next owned pick is most costly. Direct Greedy is included as a simple benchmark heuristic. Synthetic experiments evaluate point distributions, position distributions, ADP uncertainty, and large-scale stress settings, while 2026 Yahoo and FanGraphs data provide realistic validation. Opportunity Cost Greedy consistently has smaller optimality gaps than Direct Greedy and solves large synthetic instances in seconds.

1 Introduction

Fantasy baseball drafts are sequential resource-allocation problems. A manager selects players over multiple rounds, but every selection interacts with future choices: a roster position may become scarce, a flexible player may fit several slots, and a high-value player may disappear before the manager’s next pick. Average Draft Position (ADP) captures market expectations about when a player is likely to be selected. Therefore, the best projected player is not always the best decision if waiting one round causes a large expected loss at a scarce position.

This project asks whether an operations research model can provide a useful benchmark for draft quality and whether a scalable heuristic can remain close to that benchmark on larger instances. Our final comparison uses three methods. ADP-aware ILP is the exact benchmark. Direct Greedy is a simple benchmark heuristic. Opportunity Cost Greedy is the proposed scalable heuristic.

2 Problem Description

The decision maker is one team in a snake draft. Given a player pool, projected fantasy points, eligible positions, ADP values, roster requirements, and the team’s draft position, the manager must choose a roster. Each owned pick must select one available player, and every selected player must be assigned to exactly one eligible roster position. A player with ADP A_i is considered available at pick S_k only when $A_i + \delta \geq S_k$, where δ is an availability buffer.

The trade-off is between current value and future availability. Drafting the highest current projected points may ignore positional scarcity. Waiting for a position may be beneficial if strong replacements will remain, but dangerous if the position will become infeasible or much lower in value. This is why we need both an exact optimization benchmark and a heuristic that explicitly reasons about future replacement value.

3 Model Formulation

Let I be the player set, P the roster-position set, and K the set of owned picks. Let V_i be projected points, $E_{ip} \in \{0, 1\}$ eligibility, R_p required slots, A_i ADP, and S_k the overall pick number. For a snake draft with M teams and draft position j , the pick in round r is

$$S_r = \begin{cases} (r-1)M + j, & r \text{ odd}, \\ rM - j + 1, & r \text{ even}. \end{cases}$$

The binary variables are y_i , whether player i is drafted; x_{ip} , whether player i is assigned to position p ; and z_{ik} , whether player i is selected with owned pick k . The ADP-aware ILP is

$$\max \sum_{i \in I} V_i y_i \tag{1}$$

$$\text{s.t.} \quad \sum_{i \in I} z_{ik} = 1 \quad \forall k \in K, \tag{2}$$

$$\sum_{k \in K} z_{ik} = y_i \quad \forall i \in I, \tag{3}$$

$$\sum_{p \in P} x_{ip} = y_i \quad \forall i \in I, \tag{4}$$

$$x_{ip} = 0 \quad \forall i \in I, p \in P \text{ with } E_{ip} = 0, \tag{5}$$

$$\sum_{i \in I} x_{ip} = R_p \quad \forall p \in P, \tag{6}$$

$$z_{ik} = 0 \quad \forall i \in I, k \in K \text{ with } A_i + \delta < S_k, \tag{7}$$

$$x_{ip} \in \{0, 1\} \quad \forall i \in I, p \in P, \tag{8}$$

$$y_i \in \{0, 1\} \quad \forall i \in I, \tag{9}$$

$$z_{ik} \in \{0, 1\} \quad \forall i \in I, k \in K. \tag{10}$$

Constraint (2) selects one player at each owned pick. Constraint (3) links pick decisions to drafted-player decisions. Constraint (4) assigns each drafted player to one roster slot, while constraints (5) and (6) enforce eligibility and roster requirements. Constraint (7) removes player-pick pairs that are unavailable under the ADP buffer. The default roster has 16 slots: C, 1B, 2B, 3B, SS, three OF, Util, five SP, and two RP. The model is exact for the single-team planning problem under the ADP availability assumption. Its approximate variable count is $n(1 + p + r)$, where n is the number of players, p the number of roster positions, and r the roster size.

4 Algorithms

4.1 Direct Greedy: simple benchmark heuristic

Direct Greedy is intentionally simple. At each owned pick, it expires players whose $A_i + \delta$ is below the current pick. For each open position p , it computes

$$\text{scarcity ratio}_p = \frac{\text{remaining slots}_p}{\text{available eligible players}_p}.$$

It chooses the position with the largest scarcity ratio, breaking ties by the best currently available projected points, and drafts the highest-point available player for that position. It is fast and transparent, but it does not compare current value with future replacement value.

4.2 Opportunity Cost Greedy: proposed scalable heuristic

Opportunity Cost Greedy is the proposed scalable heuristic. It maintains two views of availability: players available at the current pick, and players expected to remain at the next owned pick. For each open position p , it computes

$$\text{opportunity cost}_p = \text{best current points}_p - \text{best next-pick points}_p.$$

The method drafts the player-position pair with the largest opportunity cost. It also includes a feasibility fallback. If the current or future eligible supply for a position is at or below the remaining required slots, that position receives forced priority:

$$\text{current count}_p \leq \text{remaining slots}_p \quad \text{or} \quad \text{future count}_p < \text{remaining slots}_p.$$

This prevents the heuristic from waiting too long in tight-supply cases.

Both heuristics use one max-heap per roster position, lazy deletion for expired or selected players, and active counts of available players by position. If e is the average number of eligible positions per player, the practical running time is approximately

$$O(ne \log n + rp \log n),$$

which is much smaller than constructing and solving the full ILP at large scale.

5 Data

5.1 Real data

The real-world validation instances use 2026 Yahoo and FanGraphs fantasy baseball player pools. Projection and ADP files are processed into player names, projected points, ADP values, and eligible positions. These datasets are realistic sanity checks: they show whether the synthetic findings also hold on actual player pools and scoring systems.

5.2 Synthetic data generation

Synthetic data are the main controlled experiment source. The base roster ratio follows the fantasy baseball roster above. Roster scale s multiplies all slot requirements, so $s = 1, 2, 3$ gives roster sizes 16, 32, 48. Let $D = rt$ be total league draft demand for roster size r and league size t . The player-demand ratio is n/D , with typical values 1, 3, 10. Points are generated independently from positions by default. ADP is generated from true point rank plus noise:

$$\text{ADP}_i = \text{rank}_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma_{\text{ADP}}),$$

then clipped to the player-pool range.

The point scenarios are normal, uniform, and high-low. The high-low scenario creates a small elite group and many low-value players, so missing an elite player is more costly. The position scenarios control eligibility and flexibility: uniform-by-type randomly samples hitter and pitcher patterns; point-flexible gives high-value players more eligibility; single-position removes flexibility; roster-ratio matches player position supply to roster demand.

6 Experiment Design

The report uses synthetic experiments as the main performance evaluation and real data as validation. Table 1 summarizes the scenarios.

Table 1: Synthetic experiment design.

ID	Purpose	Main controlled factors	Cases
N1	Establish a normal baseline.	normal points, roster-ratio positions, $s = 1$, $n/D = 3$, $\sigma_{\text{ADP}} = 30$, $\delta = 0$.	10 seeds
N2	Test whether value distribution changes heuristic quality.	normal, uniform, and high-low point scenarios; other factors fixed as N1.	30 cases
N3	Test whether eligibility structure and roster flexibility matter.	uniform-by-type, point-flexible, single-position, and roster-ratio position scenarios.	40 cases
N4	Test moderate scaling where ILP can still solve all cases.	roster scale $s = 1, 2, 3$ and player-demand ratio $n/D = 1, 3, 10$.	45 cases
N5	Test ADP uncertainty and delta sensitivity.	$\sigma_{\text{ADP}} = 0, 10, 30, 60, 100$ and $\delta = -10, \dots, 10$.	1050 cases
N6	Stress test scalability beyond moderate instances.	larger rosters and player pools, high-low points, single-position eligibility.	stress run

The scenarios are intentionally separated so that each experiment answers one question. N1 checks whether all methods behave reasonably in a balanced baseline. N2 changes only the point distribution, so it tests whether greedy methods fail mainly when value is concentrated among a few elite players. N3 changes only eligibility generation, so it tests whether roster flexibility helps or hurts the heuristics. N4 increases roster scale and player-pool size while keeping the design moderate enough for the ILP to solve every case, making it the cleanest comparison of runtime growth and objective quality. N5 varies ADP noise and δ , so it isolates uncertainty in draft availability. N6 combines low flexibility, high-low points, larger rosters, and larger player pools to deliberately create instances where exact optimization becomes slow or impractical.

The performance metrics are objective value, runtime, method status, Gurobi MIPGap for ILP, and heuristic optimality gap. For a heuristic solution H , the optimality gap is

$$\text{gap}(H) = \frac{\text{objective}_{\text{ILP}} - \text{objective}_H}{\text{objective}_{\text{ILP}}}.$$

This gap is computed only against ADP-aware ILP, not against a no-ADP model.

7 Results

7.1 N1 to N3: baseline, points, and positions

Table 2 shows the main quality results for N1 to N3. These experiments are designed to explain where heuristic errors come from before moving to larger scaling experiments.

Table 2: Heuristic quality in baseline, point-distribution, and position-distribution experiments.

Scenario	ILP mean objective	Opportunity Cost gap	Direct Greedy gap
N1 baseline	9228.582	1.919%	4.785%
N2 normal points	9228.582	1.919%	4.785%
N2 uniform points	11440.292	1.439%	3.243%
N2 high-low points	9137.382	2.867%	8.390%
N3 uniform-by-type	9112.331	1.074%	4.889%
N3 point-flexible	9139.980	1.344%	5.364%
N3 single-position	9070.910	0.939%	5.107%
N3 roster-ratio	9228.582	1.919%	4.785%

Opportunity Cost Greedy is consistently closer to the ADP-aware ILP than Direct Greedy. The high-low point distribution is the hardest point scenario because missing elite players is expensive. Position distribution does not create infeasibility after adding the feasibility fallback, but Direct Greedy remains more sensitive to eligibility structure.

7.2 N5: ADP uncertainty

N5 varies ADP noise and the availability buffer. Table 3 aggregates over all δ values for each noise level. As ADP becomes noisier, both heuristics generally move farther from the exact benchmark, but Opportunity Cost Greedy remains substantially closer.

Table 3: ADP uncertainty experiment. Each row aggregates 210 cases.

σ_{ADP}	ILP mean objective	Opportunity Cost gap	Direct Greedy gap
0	8449.595	0.416%	1.721%
10	8690.528	0.992%	3.342%
30	9243.851	1.800%	5.029%
60	9761.266	2.429%	5.728%
100	10047.434	2.146%	4.496%

7.3 N4 and N6: scaling

N4 confirms that both heuristics remain close to the exact benchmark while using much less runtime on moderate instances. Across 45 N4 cases, ADP-aware ILP has mean runtime 0.644 seconds and max runtime 4.915 seconds. Opportunity Cost Greedy has mean gap 1.623%, mean runtime 0.093 seconds, and max gap 3.714%. Direct Greedy has mean gap 4.177%, mean runtime 0.088 seconds, and max gap 10.349%.

N6 then extends to stress instances. Table 4 shows the scale separation. The ILP remains exact when it completes, but runtime grows sharply. At the timeout-target size, the ILP run was manually interrupted without producing benchmark output, while both heuristics solved the instances in under ten seconds.

Table 4: Runtime scaling on synthetic stress instances.

Instance	Approx. variables	ADP-aware ILP	Opportunity Cost	Direct Greedy
stress_large	2,289,600	41.131s	1.008s	0.926s
stress_xlarge	10,880,000	299.430s	2.965s	2.705s
timeout_target	49,029,120	interrupted	9.381s	8.744s

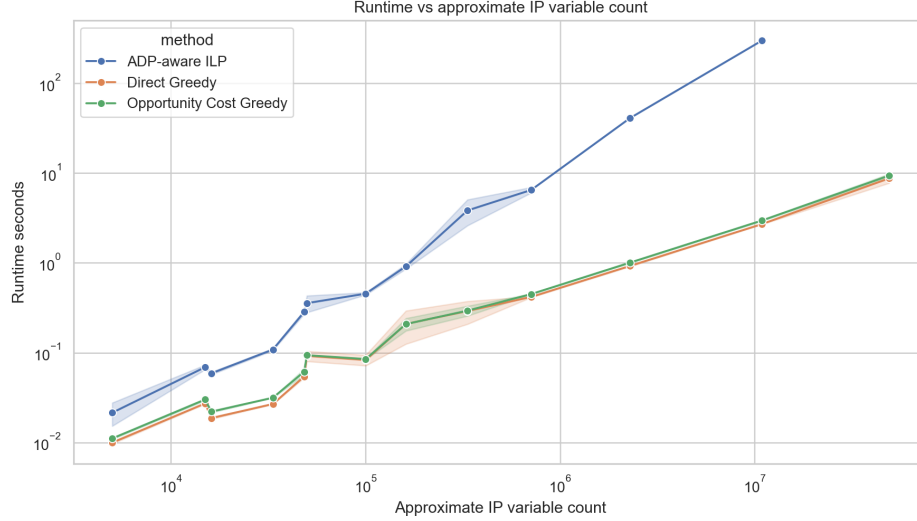


Figure 1: Runtime grows sharply for the exact benchmark, while both heuristics remain fast on large synthetic instances.

7.4 Real-data validation

The 2026 Yahoo and FanGraphs benchmarks are realistic validation cases rather than the main experiment grid. Each dataset has 252 benchmark cases. Table 5 again shows that Opportunity Cost Greedy has smaller gaps than Direct Greedy.

Table 5: 2026 real-data validation summary.

Dataset	Method	Mean objective	Mean gap	Max gap
Yahoo 2026	ADP-aware ILP	7917.209	0.000	0.000
Yahoo 2026	Opportunity Cost Greedy	7767.732	149.477	255.700
Yahoo 2026	Direct Greedy	7662.866	254.343	373.000
FanGraphs 2026	ADP-aware ILP	14642.927	0.000	0.000
FanGraphs 2026	Opportunity Cost Greedy	14320.135	322.792	508.000
FanGraphs 2026	Direct Greedy	14097.337	545.590	644.500

8 Conclusions

The ADP-aware ILP provides a precise exact benchmark for fantasy baseball draft optimization on manageable instances. Direct Greedy is useful as a simple benchmark heuristic, but it often leaves more value on

the table. Opportunity Cost Greedy, our proposed scalable heuristic, captures the value of drafting before a position deteriorates and includes a fallback for tight feasibility. Across synthetic experiments and real-data validation, it consistently improves over Direct Greedy while remaining fast.

The main implication is that exact optimization is valuable for benchmarking and small to medium instances, while a carefully designed heuristic is necessary for large-scale draft planning. Future work could add opponent-specific draft behavior, use calibrated scoring variants as robustness checks, and combine the heuristic with small local ILP subproblems for dynamic in-draft replanning.